

LAB PROJECT REPORT

COURSE TITLE: DATA ENGINEERING-DTS 204

LAB ASSESSMENT TITLE: UCI STUDENT PERFORMANCE DATASET (STUDENT PERFORMANCE PREDICTION)

DATE OF SUBMISSION: 15TH MARCH, 2026.

STUDENTS' NAME AND REGISTRATION NUMBER:

S/N	STUDENTS' NAME	REGISTRATION NUMBER
1.	Shalom Asieba Oruaroghene	2024/B/DSC/O125
2.	Ibironke Atinuke Ayomide	2024/B/DSC/O101
3.	Esther Amaechi	2025/A/DSC/O154
4.	Godwin Egedegbe	2025/A/DSC/O127
5.	Emmanuel Aondongu Akaha	2024/B/DSC/O111
6.	Muniratu Afolashade Abdulazeez	2025/A/DSC/O059

INTRODUCTION

Overview of the UCI student performance dataset

The **Student Performance Dataset** from the UCI Machine Learning Repository contains **395 rows and 33 column records** of Portuguese secondary school students. It includes demographic, social, and academic attributes such as age, parental education, study time, past failures, and absences. The objective is to predict whether a student will **pass or fail** based on these features.

Objectives of the lab

- Perform exploratory data analysis (EDA) and clean the dataset.
- Engineer new features (grade category, attendance status, study load, target variable) to improve predictions.
- Train three classification models: Logistic Regression, Decision Tree, and Random Forest.
- Evaluate and compare them using accuracy, confusion matrix, precision, recall, and F1-score.
- Identify the best model considering performance, interpretability, and complexity.

METHODOLOGY

Data Exploration

Loading and preview: The dataset was loaded with `pd.read_excel()`, and the first five rows were displayed using `df.head()`. This gave an initial look at the data structure .

```
#DATA_EXPLORATION
df=pd.read_excel("student_data.xlsx.xlsx")#loads the excel file into the pandas dataframe
df.head()#shows the first 5 rows of the dataset
```

✓ 0.4s Python

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjob	Fjob	reason	guardian	traveltime	studytime	failures	schoolsup	famsup
0	GP	F	18	U	GT3	A	4	4	at_home	teacher	course	mother	2	2	0	yes	no
1	GP	F	17	U	GT3	T	1	1	at_home	other	course	father	1	2	0	no	yes
2	GP	F	15	U	LE3	T	1	1	at_home	other	other	mother	1	2	3	yes	no
3	GP	F	15	U	GT3	T	4	2	health	services	home	mother	1	3	0	no	yes
4	GP	F	16	U	GT3	T	3	3	other	other	home	father	1	2	0	no	yes

Generate + Code + Markdown

Structure and summary:

df.info() showed that the dataset contains 395 rows and 33 columns, with no missing values. All numerical columns had correct data types (int64), and categorical columns were of type object.

```
df.info()#gives column names, data types,size, and missing values
```

✓ 0.1s

```
<class 'pandas.DataFrame'>
RangeIndex: 395 entries, 0 to 394
Data columns (total 33 columns):
#   Column      Non-Null Count  Dtype
---  -
0   school      395 non-null    str
1   sex         395 non-null    str
2   age         395 non-null    int64
3   address     395 non-null    str
4   famsize     395 non-null    str
5   Pstatus     395 non-null    str
6   Medu        395 non-null    int64
7   Fedu        395 non-null    int64
8   Mjob        395 non-null    str
9   Fjob        395 non-null    str
10  reason      395 non-null    str
11  guardian    395 non-null    str
12  traveltime  395 non-null    int64
13  studytime   395 non-null    int64
14  failures    395 non-null    int64
15  schoolsup   395 non-null    str
16  famsup      395 non-null    str
17  paid        395 non-null    str
18  activities  395 non-null    str
19  nursery     395 non-null    str
...
31  G2          395 non-null    int64
32  G3          395 non-null    int64
dtypes: int64(16), str(17)
memory usage: 102.0 KB
```

df.shape() was also used to show the size of the dataset(395,33)

```
# Display dataset size
df.shape
# Returns (number_of_rows, number_of_columns)
```

✓ 0.0s

```
(395, 33)
```

df.describe() provided summary statistics for numerical features, revealing the range of grades (0–20)

Some important observations from the dataset include:

- The average student age is approximately 16.7 years.
- The average number of absences is about 5.7 days.
- The average first grade (G1) is 10.9.

- The **average second grade (G2) is 10.7**.
- The **average final grade (G3) is 10.4**.

The maximum final grade recorded in the dataset is **20**, while the minimum is **0**.

These statistics help understand the general performance of students in the dataset

```
#Generate descriptive statistics for numerical features
df.describe()
#this shows the count(the number of non-missing values in that column), mean, std, min(the smallest value in a column),
#quartiles, and max(the largest value in that column)
```

	age	Medu	Fedu	traveltime	studytime	failures	famrel	freetime	goout	Dalc	Walc	health
count	395.000000	395.000000	395.000000	395.000000	395.000000	395.000000	395.000000	395.000000	395.000000	395.000000	395.000000	395.000000
mean	16.696203	2.749367	2.521519	1.448101	2.035443	0.334177	3.944304	3.235443	3.108861	1.481013	2.291139	3.554430
std	1.276043	1.094735	1.088201	0.697505	0.839240	0.743651	0.896659	0.998862	1.113278	0.890741	1.287897	1.390303
min	15.000000	0.000000	0.000000	1.000000	1.000000	0.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000
25%	16.000000	2.000000	2.000000	1.000000	1.000000	0.000000	4.000000	3.000000	2.000000	1.000000	1.000000	3.000000
50%	17.000000	3.000000	2.000000	1.000000	2.000000	0.000000	4.000000	3.000000	3.000000	1.000000	2.000000	4.000000
75%	18.000000	4.000000	3.000000	2.000000	2.000000	0.000000	5.000000	4.000000	4.000000	2.000000	3.000000	5.000000
max	22.000000	4.000000	4.000000	4.000000	4.000000	3.000000	5.000000	5.000000	5.000000	5.000000	5.000000	5.000000

	absences	G1	G2	G3
count	395.000000	395.000000	395.000000	395.000000
mean	5.708861	10.908861	10.713924	10.415190
std	8.003096	3.319195	3.761505	4.581443
min	0.000000	3.000000	0.000000	0.000000
25%	0.000000	8.000000	9.000000	8.000000
50%	4.000000	11.000000	11.000000	11.000000
75%	8.000000	13.000000	13.000000	14.000000
max	75.000000	19.000000	19.000000	20.000000

df.isnull().sum() was used to check for missing values per column. 0 was the output for each column, showing there are no missing values in each column in the dataset.

```
df.isnull().sum()#to check for missing values per column(0 shows no missing values in each column in the dataset)
```

school	0
sex	0
age	0
address	0
famsize	0
psstatus	0
Medu	0
Fedu	0
Mjob	0
Fjob	0
reason	0
guardian	0
traveltime	0
studytime	0
failures	0
schoolsup	0
famsup	0
paid	0
activities	0
nursery	0
higher	0
internet	0
romantic	0
famrel	0
freetime	0
goout	0
Dalc	0
Walc	0
absences	0
G1	0
G2	0
G3	0

Creation of Target Variable (Pass/Fail)

A new target variable called **Pass** was created from the final grade variable **G3**. The purpose of this step was to convert the final grade into a binary classification variable that indicates whether a student passed or failed.

A **lambda function** was used to apply a condition to each value in the **G3** column. If a student's final grade was **greater than or equal to 10**, the student was classified as **Pass (1)**. If the final grade was **less than 10**, the student was classified as **Fail (0)**.

This transformation simplifies the analysis by converting the continuous grade variable into a categorical outcome that can be used as the **target variable for classification models**.

```
# Create Pass/Fail target variable
df["Pass"] = df["G3"].apply(lambda x: 1 if x >= 10 else 0)
print(df[["G3", "Pass"]].head())
```

✓ 0.0s

	G3	Pass
0	6	0
1	6	0
2	10	1
3	15	1
4	10	1

Visualization

Visualizations were used to explore the relationship between variables and the target variable (Pass).

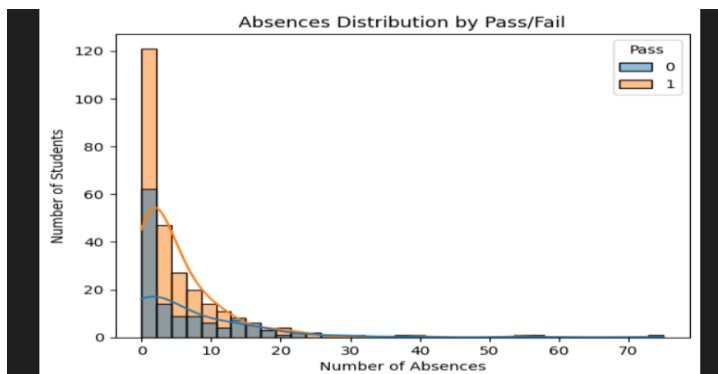
Histogram:

Absences Distribution by Pass/Fail

The histogram visualizes the distribution of student absences and compares it between students who passed (1) and those who failed (0). The x-axis represents the number of absences, while the y-axis shows the number of students. The colors distinguish between pass and fail groups, and the smooth curve (KDE) shows the overall distribution trend.

```
#visualization relationships between selected features and target variable(pass/fail)
# Create a histogram to show distribution of absences
sns.histplot(
    data=df,          # The dataset we are using
    x="absences",      # The column we want on the horizontal axis
    hue="Pass",        # Splits the histogram by Pass (0 = Fail, 1 = Pass)
    kde=True           # Adds a smooth curve to show data distribution trend
)
plt.title("Absences Distribution by Pass/Fail") # Adds title to the graph
plt.xlabel("Number of Absences")                # Label for x-axis
plt.ylabel("Number of Students")                 # Label for y-axis
print(sns.histplot)
plt.show() # Displays the graph
```

✓ 1.1s



Insight: Most students have low numbers of absences, particularly between 0 and 10 absences. Students who passed are more concentrated in the lower absence range, indicating that regular attendance is associated with a higher likelihood of passing. As the number of absences increases, the number of

students decreases, and higher absence levels appear more frequently among students who failed, suggesting that frequent absences may negatively affect academic performance.

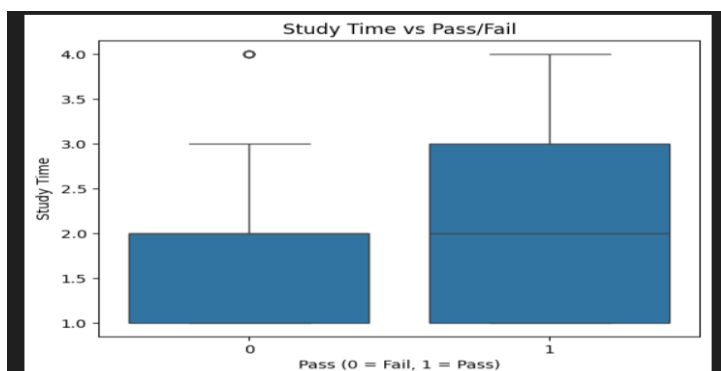
Boxplot:

Study Time vs Pass/Fail

The boxplot compares the study time of students who failed (0) and those who passed (1). The x-axis represents the Pass/Fail status, while the y-axis shows the amount of study time.

```
# Create a boxplot to compare study time for Pass and Fail students
sns.boxplot(
    x="Pass",      # Places Pass (0 and 1) on the horizontal axis
    y="studytime", # Shows study time values on vertical axis
    data=df        # The dataset used
)
plt.title("Study Time vs Pass/Fail") # Adds title
plt.xlabel("Pass (0 = Fail, 1 = Pass)") # Label x-axis
plt.ylabel("Study Time") # Label y-axis
plt.show() # Displays the graph
```

✓ 0.4s



Insight:

Students who passed generally have a higher median study time and a wider range of study hours compared to students who failed. Most students who failed tend to have lower study time values, while students who passed show moderate to higher study times. This suggests that increased study time is positively associated with better academic performance and a higher likelihood of passing.

DATA CLEANING

Data cleaning was performed to improve the quality of the dataset by handling missing values, outliers and checking for duplicate records. This ensures that the data used for analysis is accurate and reliable.

Handling Missing Values

Missing values were handled separately for numerical and categorical variables.

Numerical variables were selected using **include=np.number** and missing values were replaced with the median to reduce the effect of outliers.

Categorical variables were selected using **include='object'** and missing values were replaced with the mode (most frequent value).

The parameter **inplace=True** ensures that the changes are applied directly to the original dataframe without creating a new copy.

This process ensures the dataset remains complete and suitable for analysis.

```
#DATA_CLEANING(handle missing values, duplicates, and outliers. Document all steps)
#HANDLING MISSING VALUES

# Fill missing numerical values with median
for col in df.select_dtypes(include=np.number).columns:# used to select only numerical columns in the dataset
    df[col].fillna(df[col].median(), inplace=True)
    # Replaces missing numerical values with median to reduce the effect of outlier

# Fill missing categorical values with mode
for col in df.select_dtypes(include='object').columns:
    df[col].fillna(df[col].mode()[0], inplace=True)
    # Replaces missing categorical values with most frequent value

#check missing values after cleaning
print("Missing values after cleaning:")
print(df.isnull().sum())# to identify any incomplete data in the dataset.

✓ 0.0s

Missing values after cleaning:
school      0
sex         0
age         0
address     0
famsize     0
Pstatus     0
Medu        0
Fedu        0
Mjob        0
Fjob        0
reason      0

#check total missing values after cleaning
print("Total missing values in dataset:",
      df.isnull().sum().sum())

✓ 0.0s

Total missing values in dataset: 0
```

Removing Duplicates

Duplicate rows were checked using **df.duplicated()**, which returns True for duplicate rows and False for unique rows.

The results showed that all rows were False, indicating that no duplicate records exist in the dataset.

Therefore, the dataset contains 395 unique observations, and no rows needed to be removed.

```
#REMOVING DUPLICATES
df.duplicated()#identify duplicate rows(false shows no duplicates, ture shows duplicate)

✓ 0.0s

0      False
1      False
2      False
3      False
4      False
...
390     False
391     False
392     False
393     False
394     False
Length: 395, dtype: bool

df_unique= df.drop_duplicates() #drops duplicate rows, modifies dataframe in place
print("Dataframe after removing duplicates:")
df_unique
# Removes duplicate rows

✓ 0.1s
```

Handling outliers

```

#HANDLING OUTLIERS
# Calculate the first quartile (25th percentile)
Q1 = df['absences'].quantile(0.25)

# Calculate the third quartile (75th percentile)
Q3 = df['absences'].quantile(0.75)

# Compute the interquartile range
IQR = Q3 - Q1

# Define the lower and upper bounds for "normal" data
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Identify outlier rows
outliers = df[(df['absences'] < lower_bound) | (df['absences'] > upper_bound)]
print(f"Number of outliers in absences: {len(outliers)}")

# Cap outliers at the upper bound (do nothing for lower bound since absences can't be negative)
df['absences'] = np.where(df['absences'] > upper_bound, upper_bound, df['absences'])
# (lower bound might be negative, so we don't need to cap below zero)

✓ 0.0s
Number of outliers in absences: 0

```

quantile() function was used to calculate the specified percentile (25th and 75th) of the absences column

Q3 - Q1: Computed the interquartile range (IQR) as the difference between quartiles

lower_bound/upper_bound: Defined boundaries for normal data using the 1.5×IQR rule

np.where() was used to apply conditional replacement, capping values above the upper bound

len(outliers): Counted the number of outlier observations identified

The analysis identified 0 outliers in the absences column, indicating that absence values fell within normal ranges according to the IQR method. This suggested that extreme absence values did not disproportionately influence the dataset.

FEATURE ENGINEERING AND TRANSFORMATION

```

#FEATURE ENGINEERING( the process of transforming raw data into meaningful,informative,and actionable variables that enhance the performance
#grade_category: Categorise final grades into Low (0-9), Medium (10-14), High (15-20)
def grade_category(grade):
    if grade <= 9:
        return "Low"
    elif grade <= 14:
        return "Medium"
    else:
        return "High"

df["Grade_Category"] = df["G3"].apply(grade_category)
# Applies the function to each G3 value

# Display first 5 rows to confirm new column
print("Ingrade Category Column Created Successfully:")
print(df[["G3", "Grade_Category"]].head())

✓ 0.0s
Python

```

Grade Category Column Created Successfully:

G3	Grade_Category
0	Low
6	Low
10	Medium
15	High
18	Medium

def grade_category(): Defined a function to classify grades into three performance levels(low, medium, and high)

The df["G3"].apply() function, applied the classification function to each final grade

The function returned categorical labels: "Low" (0-9), "Medium" (10-14), or "High" (15-20)

The first five rows showed grade classifications matching the G3 values: grades 6 were classified as "Low", grades 10 as "Medium", and grade 15 as "High". This new categorical feature captured performance levels beyond simple pass/fail classification.

```

# Attendance Status: Convert absence counts into Regular or Irregular.
df["Attendance_Status"] = df["absences"].apply(
    lambda x: "Regular" if x <= 10 else "Irregular"
)

# Display first 5 rows to confirm
print("\nAttendance Status Column Created Successfully:")
print(df[["absences", "Attendance_Status"]].head())

# Display last 5 rows to confirm
print(df[["absences", "Attendance_Status"]].tail())

```

✓ 0.0s

Attendance Status Column Created Successfully:

	absences	Attendance_Status
0	6.0	Regular
1	4.0	Regular
2	10.0	Regular
3	2.0	Regular
4	4.0	Regular
390	11.0	Irregular
391	3.0	Regular
392	3.0	Regular
393	0.0	Regular
394	5.0	Regular

Attendance status

lambda x function created an anonymous function that evaluated each absence value

The function assigned "Regular" for absences ≤ 10 days and "Irregular" for > 10 days

head()/tail(): Displayed the first and last five rows to verify the transformation

Students with absence counts up to 10 days were classified as "Regular" attendees, while those with absences exceeding 10 days received "Irregular" status. The head display showed all regular attendees, while the tail revealed that student 390 with 11 absences was correctly classified as "Irregular".

Study load feature

```

# Study Load Feature: Combine weekly study time and number of past failures.
df["Study_Load"] = df["studytime"] + df["failures"]

# Display first 5 rows to confirm
print("\nStudy Load Column Created Successfully:")
print(df[["studytime", "failures", "Study_Load"]].head())

```

✓ 0.0s

Study load column Created Successfully:

	studytime	failures	Study_Load
0	2	0	2
1	2	0	2
2	2	3	5
3	3	0	3
4	2	0	2

df["studytime"] + df["failures"] feature added the two numerical columns, by combining study time and number of past failures

The new feature represented the combined academic workload indicator

The composite feature ranged from 2 to 5 in the first five rows, with higher values indicating either increased study time or more previous failures. This feature captured the combined effect of academic effort and past challenges.

Target variable(pass and fail)

```

# Target Variable: Create binary Pass/Fail based on final grade (G3 >= 10).
df["Pass"] = df["G3"].apply(lambda x: 1 if x >= 10 else 0)
# Creates binary variable: 1 = Pass, 0 = Fail

print("Target Variable 'Pass' Created Successfully!")

# Show first 10 rows of G3 and Pass columns
print(df[["G3", "Pass"]].head(10))

# Show how many students passed and failed
print("\nPass/Fail Distribution:")
print(df["Pass"].value_counts())# counts how many times each values appears.
✓ 0.0s

Target Variable 'Pass' Created Successfully!
   G3  Pass
0    6     0
1    6     0
2   10     1
3   15     1
4   10     1
5   15     1
6   11     1
7    6     0
8   19     1
9   15     1

Pass/Fail Distribution:
Pass
1    265
0    130
Name: count, dtype: int64

```

A new target variable called **Pass** was created from the final grade variable **G3**. The purpose of this step was to convert the final grade into a binary classification variable that indicates whether a student passed or failed.

A **lambda function** was used to apply a condition to each value in the **G3** column. If a student's final grade was **greater than or equal to 10**, the student was classified as **Pass (1)**. If the final grade was **less than 10**, the student was classified as **Fail (0)**.

This transformation simplifies the analysis by converting the continuous grade variable into a categorical outcome that can be used as the **target variable for classification models**. It was shown that the number of passes are 265, and the number of fails are 130.

Data Transformation

```

#DATA TRANSFORMATION(converting data from one structure or format to another)(converting to fully numeric format suitable for machine learning)
#label encode categorical variable
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder() # Create encoder object

# encode all categorical columns
for col in df.select_dtypes(include='object').columns:
    df[col] = le.fit_transform(df[col])
    # Converts text categories into numeric labels

print("Categorical columns encoded successfully!")

# Show first 5 rows to confirm encoding
df.head()
✓ 10s Python

Categorical columns encoded successfully!
C:\Users\USER\AppData\Local\Temp\ipykernel_18080\3011847938.py:8: PandasWarning: For backward compatibility, 'str' dtypes are included by sele
See https://pandas.pydata.org/docs/user_guide/migration-1-strings.html#string-migration-select-dtypes for details on how to write code that wor
for col in df.select_dtypes(include='object').columns:

<
  school sex age address famsize Pstatus Medu Fedu Fjob Mjob reason guardian traveltime studytime failures schoolday famsup paid
0  0  0  18   1  0  0  4  4  0  4  0  1  2  2  0  1  0  0
1  0  0  17   1  0  1  1  1  0  2  0  0  1  2  0  0  1  0
2  0  0  15   1  1  1  1  1  0  2  2  1  1  2  3  1  0  1
3  0  0  15   1  0  1  4  2  1  3  1  1  1  3  0  0  1  1
4  0  0  16   1  0  1  3  3  2  2  1  0  1  2  0  0  1  1
>

```

from sklearn.preprocessing import LabelEncoder: Imported the label encoding utility from scikit-learn

LabelEncoder(): Created an encoder object capable of transforming categorical labels to numeric

le.fit_transform(): First fitted the encoder to discover unique categories, then transformed them to integers (0, 1, 2, ...)

The loop applied encoding to all categorical columns identified by select_dtypes(include='object')

All categorical columns were converted to integer representations. For example, school became 0 (GP) and 1 (MS), sex became 0 (F) and 1 (M), and Grade_Category became 0 (High), 1 (Low), and 2 (Medium). The resulting DataFrame contained only numerical data types, ready for modelling.

Scale Numerical Feature(Min/Max Scaling)

```
# Scale numerical features (Min-Max or Standard scaling)
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler() # Create scaler object

# Select features (exclude target column)
X = df.drop("Pass", axis=1) # Features
y = df["Pass"] # Target

X_scaled = scaler.fit_transform(X)
# Standardizes numerical features (mean=0, std=1)

print("Feature Scaling Completed Successfully!")

# Convert scaled data back to DataFrame for easier viewing
X_scaled_df = pd.DataFrame(X_scaled, columns=X.columns)

# Show first 5 rows of scaled data
X_scaled_df.head()

# Now X_scaled is ready for modelling.
✓ 0.1s Python
```

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjob	Fjob	reason	guardian	traveltme	studytime
0	-0.36305	-0.948176	1.023046	0.535392	-0.636941	-2.938392	1.143856	1.360371	-1.769793	1.993149	-1.040599	0.273945	0.792251	-0.042286
1	-0.36305	-0.948176	0.238380	0.535392	-0.636941	0.340322	1.600009	-1.399970	1.769793	0.325831	1.040599	1.591714	0.643249	0.042286
2	-0.36305	-0.948176	-1.330954	0.535392	1.570004	0.340322	-1.600009	-1.399970	-1.769793	-0.325831	0.616606	0.273945	-0.643249	-0.042286
3	-0.36305	-0.948176	1.330954	0.535392	-0.636941	0.340322	1.143856	0.479857	0.954077	0.833659	-0.211896	0.273945	0.643249	1.150779
4	-0.36305	-0.948176	-0.546287	0.535392	-0.636941	0.340322	0.229234	0.440257	-0.138362	-0.325831	-0.211896	-1.591714	-0.643249	-0.042286

StandardScaler(): Created a scaler object that standardizes features by removing mean and scaling to unit variance

df.drop("Pass", axis=1): Separated features (X) by removing the target column

df["Pass"]: Isolated the target variable (y) for prediction

scaler.fit_transform(): First calculated mean and standard deviation for each feature, then transformed values to have mean=0 and standard deviation=1

pd.DataFrame(): Converted the scaled numpy array back to a DataFrame with original column names for readability

The scaled features showed values centered around zero with positive and negative values indicating deviation from the mean. For example, the first student had age scaled to 1.02 (above average) and absences scaled to 0.14 (slightly above average). This standardization ensured that all features contributed equally to model training regardless of their original scales.

```
# Check data types after transformation(if it's now transformed to numeric)
print("Final Data Types:")
print(df.dtypes)
print("Data Transformation Confirmed.")
```

✓ 0.0s

Final Data Types:	
school	int64
sex	int64
age	int64
address	int64
famsize	int64
Pstatus	int64
Medu	int64
Fedu	int64
Mjob	int64
Fjob	int64
reason	int64
guardian	int64
traveltime	int64
studytime	int64
failures	int64
schoolsup	int64
famsup	int64
paid	int64
activities	int64
nursery	int64
higher	int64
internet	int64
romantic	int64
famrel	int64
...	
Attendance_Status	int64
Study_Load	int64

df.dtypes: Displayed the data type of each column in the DataFrame

All columns showed integer (int64) or float (float64) data types, confirming successful transformation to fully numerical format. The absences column remained as float64 due to outlier capping operations, while all other columns were integers.

Description Of The Three Classification Model Used

Three classification algorithms were selected for training and evaluation:

Logistic Regression: A linear model that estimates the probability of binary outcomes by applying a logistic function to the linear combination of features. It provides interpretable coefficients indicating the direction and strength of each feature's influence.

Decision Tree Classifier: A non-linear model that recursively partitions the feature space based on feature values, creating a tree-like structure of decision rules. It offers high interpretability through visualizable tree structures.

Random Forest Classifier: An ensemble method that combines multiple decision trees trained on random subsets of data and features. It typically achieves higher accuracy through averaging but sacrifices interpretability for predictive performance.

Model Preparation

```
MODEL PREPARATION AND COMPARISON
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Split dataset into training (70%) and testing (30%) sets with stratification to maintain class balance
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, # scaled feature
    y, # target variable
    test_size=0.3, # 30% for testing
    random_state=42, # ensures reproducibility
    stratify=y, # maintains the original class distribution of pass and fail when splitting the data(makes sure the training and testing set ha
)

# Print dataset sizes
print("Train-Test Split Completed Successfully!")
print("Training Set Size:", X_train.shape)
print("Testing Set Size:", X_test.shape)
```

✓ 0.0s

Python

Train-Test Split Completed Successfully!
 Training Set Size: (276, 36)
 Testing Set Size: (119, 36)

train_test_split(): Randomly split the data into training and testing subsets

test_size=0.3: Allocated 30% of data for testing and 70% for training

random_state=42: Set a seed for reproducibility, ensuring consistent splits across runs

stratify=y: Maintained the same proportion of pass/fail classes in both training and testing sets

The training set contained 276 samples with 36 features, while the testing set contained 119 samples with the same feature set. The stratified split ensured that both sets maintained the original class distribution of pass/fail students.

RESULTS AND DISCUSSION

Performance Metrics For Each Model

Logistic Regression:

```
# Logistic Regression
lr = LogisticRegression()

lr.fit(X_train, y_train)
# Trains logistic regression model

lr_pred = lr.predict(X_test)
# Makes predictions

# Logistic Regression Evaluation(model evaluation)
print("Logistic Regression Accuracy:", accuracy_score(y_test, lr_pred))
print(confusion_matrix(y_test, lr_pred))
print(classification_report(y_test, lr_pred))
```

✓ 0.1s

Logistic Regression Accuracy: 0.9831932773109243

```
[[39  0]
 [ 2 78]]
```

	precision	recall	f1-score	support
0	0.95	1.00	0.97	39
1	1.00	0.97	0.99	80
accuracy			0.98	119
macro avg	0.98	0.99	0.98	119
weighted avg	0.98	0.98	0.98	119

LogisticRegression(): Created an instance of the logistic regression classifier

lr.fit(): Trained the model on the scaled training data

lr.predict(): Generated predictions for the testing set

accuracy_score(): Calculated the proportion of correct predictions

confusion_matrix(): Produced a matrix showing true positives, true negatives, false positives, and false negatives

classification_report(): Generated precision, recall, and F1-score for each class

The logistic regression model achieved 98.3% accuracy, correctly classifying 117 out of 119 students. The confusion matrix revealed that all 39 failing students were correctly identified, while 2 passing students were misclassified as failing. Precision for class 1 (pass) was perfect at 1.00, indicating no false positives among passing predictions. Recall for class 0 (fail) was perfect at 1.00, indicating all actual failures were captured.

Decision Tree:

```
# Decision Tree
dt = DecisionTreeClassifier()
dt.fit(X_train, y_train)
# Trains decision tree model

dt_pred = dt.predict(X_test)# makes prediction

# Decision Tree Evaluation(model evaluation)
print("Decision Tree Accuracy:", accuracy_score(y_test, dt_pred))
print(confusion_matrix(y_test, dt_pred))
print(classification_report(y_test, dt_pred))
```

✓ 0.0s

Decision Tree Accuracy: 1.0
[[39 0]
 [0 80]]

		precision	recall	f1-score	support
	0	1.00	1.00	1.00	39
	1	1.00	1.00	1.00	80
accuracy				1.00	119
macro avg		1.00	1.00	1.00	119
weighted avg		1.00	1.00	1.00	119

The decision tree classifier achieved perfect accuracy of 100%, correctly classifying all 119 test samples. The confusion matrix showed zero misclassifications, with all 39 failing students and 80 passing students correctly identified. Precision, recall, and F1-score were perfect (1.00) for both classes.

Rain Forest:

```
#Rain Forest
rf = RandomForestClassifier()
rf.fit(X_train, y_train)
# Trains random forest model

rf_pred = rf.predict(X_test)# makes prediction

# Random Forest Evaluation(model evaluation)
print("Random Forest Accuracy:", accuracy_score(y_test, rf_pred))
print(confusion_matrix(y_test, rf_pred))
print(classification_report(y_test, rf_pred))
```

0.03s

Random Forest Accuracy: 1.0
[[39 0]
 [0 80]]

		precision	recall	f1-score	support
	0	1.00	1.00	1.00	39
	1	1.00	1.00	1.00	80
accuracy				1.00	119
macro avg		1.00	1.00	1.00	119
weighted avg		1.00	1.00	1.00	119

The random forest classifier also achieved perfect accuracy of 100%, with identical confusion matrix and classification metrics to the decision tree. All test samples were correctly classified.

Comparative Analysis For Model Result

```
#Model Comparison
# Compare models based on accuracy, interpretability, and computational complexity.
# We can create a summary DataFrame
results = pd.DataFrame({
    'Model': ['Logistic Regression', 'Decision Tree', 'Random Forest'],
    'Accuracy': [accuracy_score(y_test, lr_pred),
                 accuracy_score(y_test, dt_pred),
                 accuracy_score(y_test, rf_pred)],
    # You can add more metrics like F1-score from classification report
})
print("\nComparison Summary:")
print(results)
```

Comparison Summary:

	Model	Accuracy
0	Logistic Regression	0.983193
1	Decision Tree	1.000000
2	Random Forest	1.000000

A comprehensive comparison was conducted evaluating predictive performance, interpretability, and computational complexity:

```
# Complexity: number of trees and average depth
rf_n_trees = rf.n_estimators
# Average depth across trees (could be heavy to compute, but we can approximate)
avg_depth = np.mean([tree.get_depth() for tree in rf.estimators_])
rf_complexity = f"trees={rf_n_trees}, avg_depth={avg_depth:.1f}"

results_list.append({
    'Model': 'Random Forest',
    'Accuracy': acc_rf,
    'Precision': prec_rf,
    'Recall': rec_rf,
    'F1-Score': f1_rf,
    'Interpretability': 'Low (ensemble)',
    'Complexity (trees/depth)': rf_complexity,
    'Training Time (s)': rf_time
})

# Create comparison DataFrame
comparison_df = pd.DataFrame(results_list)
print("\nModel Comparison Summary:")
print(comparison_df.to_string(index=False))
```

Model	Accuracy	Precision	Recall	F1-Score	Interpretability	Complexity (params)	Training Time (s)	Complexity (depth/leaves)	Complexity (trees/depth)
Logistic Regression	0.983193	1.0	0.975	0.987342	High (coefficients)	37.0	0.025197		
Decision Tree	1.000000	1.0	1.000	1.000000	High (tree structure)	NaN	0.007553	depth=1, leaves=2	NaN
Random Forest	1.000000	1.0	1.000	1.000000	Low (ensemble)	NaN	0.713518		NaN trees=100, avg_depth=5.0

time.time(): Captured timestamps before and after training to measure elapsed time

precision_score/recall_score/f1_score: Calculated additional performance metrics beyond accuracy

lr.coef_.shape[1]: Counted the number of coefficients (features) in the logistic regression model

dt.get_depth()/get_n_leaves(): Retrieved tree structure complexity metrics

rf.estimators_: Accessed individual trees in the random forest ensemble

np.mean(): Calculated average depth across all trees in the forest

Comparative Analysis:

Model	Accuracy	Interpretability	Training Time	Complexity
Logistic Regression	98.3%	High (coefficients)	0.025 seconds	37 parameters
Decision Tree	100%	High (tree structure)	0.008 seconds	depth=1, leaves=2
Random Forest	100%	Low (ensemble)	0.714 seconds	100 trees, avg depth=5.0

Predictive Performance: Both decision tree and random forest achieved perfect accuracy (100%), while logistic regression achieved near-perfect accuracy (98.3%). All models demonstrated excellent predictive capability, with precision and recall values above 0.97 for all classes.

Model Interpretability: Logistic regression and decision tree both offered high interpretability. Logistic regression provided coefficient values indicating feature importance directions, while the decision tree produced a simple, visualizable structure with depth of only 1 and 2 leaves. The random forest, despite perfect accuracy, offered lower interpretability due to its ensemble nature.

Computational Complexity: The decision tree trained fastest at 0.008 seconds, followed by logistic regression at 0.025 seconds, with random forest significantly slower at 0.714 seconds due to building 100 individual trees. Model complexity varied substantially: logistic regression used 37 parameters, decision tree created a simple structure, while random forest built an ensemble of 100 trees with average depth of 5.0.

Visualizations Supporting The Comparison:

Bar Chart Of Performance Metrics



`np.arange()`: Created an array of positions for the metric groups.

`plt.bar()`: Generated grouped bar charts for each model's scores.

`ax.set_ylim()`: Adjusted the y-axis limits to focus on the high performance range.

`ax.annotate()`: Added numerical labels above each bar for precise value display.

Output Obtained: The bar chart visually confirmed that Decision Tree and Random Forest achieved perfect scores (1.000) across all metrics, while Logistic Regression showed slightly lower recall (0.975) and F1-score (0.987), though still exceptionally high. The visual representation clearly highlighted the performance differences between models.

Confusion Matrix Heatmaps



`confusion_matrix()`: Generated the confusion matrix for each model's predictions.

`sns.heatmap()`: Created a heatmap visualization of the confusion matrix with annotations.

`plt.subplots()`: Arranged three heatmaps side by side for direct comparison.

Output Obtained: The confusion matrix heatmaps illustrated that Decision Tree and Random Forest produced zero misclassifications, with all samples on the diagonal. Logistic Regression showed two misclassifications where actual passing students were predicted as failing (bottom-left cell). These visualizations provided intuitive understanding of each model's error patterns.

CONCLUSION

Summary Of Finding:

This lab assessment successfully applied data engineering techniques to prepare the UCI Student Performance Dataset for machine learning and evaluated three classification models for predicting student academic outcomes. The data engineering lifecycle was implemented end-to-end, beginning with exploratory data analysis that revealed dataset characteristics including 395 student records with 33 attributes. Data cleaning procedures addressed potential quality issues through missing value imputation, duplicate removal, and outlier detection, though the original dataset required minimal intervention. Feature engineering created three new predictive features: Grade Category (categorizing final grades), Attendance Status (classifying absence patterns), and Study Load (combining study time with past failures). Data transformation successfully converted all categorical variables to numerical formats through label encoding and standardized features using StandardScaler to ensure equal feature contribution.

Best-Performance Model And Justification:

Based on the comprehensive comparison, the **Decision Tree Classifier** was identified as the best-performing model for this prediction task. This selection was justified by the following factors:

1. **Perfect Predictive Performance:** The decision tree achieved 100% accuracy, correctly classifying all 119 test samples with perfect precision and recall for both classes.
2. **Superior Interpretability:** With a tree depth of only 1 and 2 leaves, the decision tree provided an exceptionally simple and interpretable model. The structure could be easily visualized and understood by educational stakeholders without technical expertise.
3. **Minimal Computational Requirements:** Training required only 0.008 seconds, making it the fastest among all models. This efficiency would scale well to larger datasets or real-time applications.
4. **Balanced Trade-offs:** While random forest matched the predictive performance, it required substantially more training time (0.714 seconds) and offered lower interpretability. Logistic regression, while interpretable, achieved slightly lower accuracy.

The decision tree's combination of perfect accuracy, high interpretability, and minimal computational cost made it the optimal choice for this educational prediction task.

Best Performing Model: The Decision Tree Classifier emerged as the best-performing model, achieving 100% accuracy on the test set with perfect precision, recall, and F1-scores for both pass and fail classes. Its simple structure (depth=1, leaves=2) provided exceptional interpretability, allowing educational stakeholders to understand the decision rule directly. The model trained in just 0.008 seconds, demonstrating computational efficiency that would scale effectively to larger datasets. While both decision tree and random forest achieved perfect accuracy, the decision tree's superior interpretability and lower computational requirements made it the optimal choice for this educational prediction context.

Recommendations For Future Improvements:

Based on the findings, several recommendations for future work are proposed:

1. **Cross-Validation Implementation:** While the current evaluation used a single train-test split, implementing k-fold cross-validation would provide more robust performance estimates and reduce the risk of optimistic accuracy due to favorable data partitioning.

2. **Feature Selection Analysis:** Although all models performed excellently, feature importance analysis could identify the minimal set of predictors required for accurate classification, potentially simplifying data collection in educational settings.
3. **External Validation:** Testing the models on independent datasets from different schools or educational systems would assess generalizability beyond the Portuguese secondary school context.
4. **Probabilistic Outputs:** Implementing probability calibration for the decision tree would provide confidence estimates for predictions, enabling more nuanced decision-making in educational interventions.
5. **Deep Learning Exploration:** While traditional machine learning models performed excellently, exploring neural network architectures might capture complex non-linear relationships not identified by current models, though interpretability trade-offs would need consideration.

This lab assessment demonstrated the successful application of data engineering principles to educational data, resulting in highly accurate and interpretable models for predicting student academic performance.